LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 4

Oleh :

M.YAZEM AGVA ROIZ

NIM 2511533003

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2026

Penulis

DAFTAR ISI

BAB 1.....4
1. Latar Belakang.....4
2. Tujuan.....4
3. Manfaat.....4
BAB 2.....5
A. QueueArray_2511533003 dan QueueArrayDriver_2511533003.....5
B. QueueLinkedList_2511533003.....9
C. ReverseData_2511533003.....10
D. IterasiQueue_2511533003.....12
E. Tugas.....13
BAB 3.....25
a. Kesimpulan.....25
b. Saran.....25

BAB 1

PENDAHULUAN

1. Latar Belakang

Queue merupakan implementasi ADT yang menggunakan konsep FIFO (First In First Out). konsep ini memungkin menyimpan data dan mengambilnya berdasarkan urutan.

Struktur queue sudah sangatlah terkenal dalam menyelesaikan masalah atau algoritma yang sangat mementingkan urutan dan implementasinya sudah sangat luas salah satu nya pada Java yang telah menyediakan Queue pada standar librarynya.

2. Tujuan

Tujuan dilakukan praktikum ini adalah melatih kemampuan dengan code code yang memiliki kasus kasus tertentu.

3. Manfaat

Manfaat dalam melakukan praktikum ini adalah mendapatkan kemampuan dan pemahaman yang dalam pada konsep queue ini dengan cara membuat code yang memiliki kasus penggunaan masing masing.

BAB 2

PEMBAHASAN

A. QueueArray_2511533003 dan QueueArrayDriver_2511533003

```
package pekan4_2511533003;
public class QueueArray_2511533003 {
    int front_3003, rear_3003, size_3003;
    int capacity_3003;
    int array_3003[];
    public QueueArray_2511533003(int capacity_3003){
        this.capacity_3003 = capacity_3003;
        front_3003 = this.size_3003 = 0;
        rear_3003 = capacity_3003 - 1;
        array_3003 = new int[this.capacity_3003];
    }
    boolean isFull_3003(QueueArray_2511533003 queue_3003){
        return (queue_3003.size_3003 == queue_3003.capacity_3003);
    }
    boolean isEmpty_3003(QueueArray_2511533003 queue_3003){
        return (queue_3003.size_3003 == 0);
    }
    void enqueue_3003(int item_3003){
        if (isFull_3003(this)) return;
        this.rear_3003 = (this.rear_3003 + 1) % this.capacity_3003;
        this.array_3003[this.rear_3003] = item_3003;
        this.size_3003 = this.size_3003 + 1;
```

```
    System.out.println(item_3003 + " enqueue to queue");
}
int dequeue_3003(){
    if (isEmpty_3003(this)) return Integer.MIN_VALUE;
    int item_3003 = this.array_3003[this.front_3003];
    this.front_3003 = (this.front_3003 + 1) % this.capacity_3003;
    this.size_3003 = this.size_3003 - 1;
    return item_3003;
}
int front_3003(){
    if (isEmpty_3003(this)) return Integer.MIN_VALUE;
    return this.array_3003[this.front_3003];
}
int rear_3003(){
    if (isEmpty_3003(this)) return Integer.MIN_VALUE;
    return this.array_3003[this.rear_3003];
}
void display_3003(){
    int i_3003;
    if(this.front_3003 == this.rear_3003){
        System.out.println("\nAntrian Kosong\n");
        return;
    }
    for (i_3003 = this.front_3003; i_3003 < rear_3003; i_3003++){
        System.out.printf(" %d <-- ", array_3003[i_3003]);
```

```
        }
        return;
    }
}
```

Code diatas merupakan ADT QueueArray_2511533003 yang berguna untuk membuat struktur Queue yang memungkinkan FIFO. memiliki method antara lain:

1. isFull_3003

   Merupakan method yang berfungsi untuk mengetahui apakah queue penuh.

2. isEmpty_3003

   Merupakan method yang berfungsi untuk mengetahui apakah queue kosong.

3. enqueue_3003

   Method berfungsi untuk memasukkan data ke dalam queue.

4. dequeue_3003

   Method berfungsi untuk mengeluarkan data ke dalam queue.

5. front_3003

   Method berfungsi untuk melihat queue di depan.

6. rear_3003

   Method berfungsi untuk melihat queue dibelakang.

7. display_3003

   Menampilkan queue menggunakan iterasi looping.

Code di atas masihlah ADT, oleh karena itu diperlukan code Driver yaitu QueueArrayDriver_2511533003

```
package pekan4_2511533003;

public class QueueArrayDriver_2511533003 {
    public static void main(String[] args) {
        QueueArray_2511533003 queue_3003 = new QueueArray_2511533003(1000);
        queue_3003.enqueue_3003(10);
        queue_3003.enqueue_3003(20);
        queue_3003.enqueue_3003(30);
        queue_3003.enqueue_3003(40);

        System.out.println("item di dapan " + queue_3003.front_3003());
        System.out.println("item paling di belakang " + queue_3003.rear_3003());
        System.out.println("tampilkan queue");
        queue_3003.display_3003();

        System.out.println();
        System.out.println(queue_3003.dequeue_3003() + " dihapus dari queue");
        System.out.println("item di dapan: " + queue_3003.front_3003());
        System.out.println("item dibelakang " + queue_3003.rear_3003());
        System.out.println("tampilkan queue setelah satu data dihapus");
        queue_3003.display_3003();
    }
}
```

Code diatas merupakan code Driver yang menggunakan ADT QueueArray_2511533003 yang mana alur codenya adalah:

1. Inisialisasi ADT QueueArray_2511533003
2. Memasukkan nilai ke QueueArray_2511533003
3. Menggunakan method yang telah disediakan oleh ADT QueueArray_2511533003

B. QueueLinkedList_2511533003

```
package pekan4_2511533003;

import java.util.LinkedList;

import java.util.Queue;


public class QueueLinkedList_2511533003 {


    public static void main(String[] args) {

        Queue<Integer> q_3003 = new LinkedList<>();

        // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian

        for (int i = 0; i < 6; i++) q_3003.add(i);


        // Menampilkan ISi Antrian

        System.out.println("Elemen Antrian " + q_3003);

        int hapus_3003 = q_3003.remove();


        // Untuk menghapus kepala antrian

        System.out.println("Hapus Elemen = " + hapus_3003);

        System.out.println(q_3003);
```

```
        // Untuk Melihat Antrian Terdepan
        int depan_3003 = q_3003.peek();
        System.out.println("Kepala Antrian " + depan_3003);

        int banyak_3003 = q_3003.size();
        System.out.println("Size Anterian = " + banyak_3003);
    }
}
```

Code diatas menggunakan interface Queue yang telah disediakan oleh Java pada standar librarynya, dengan Queue ini membuat dalam pengembangan aplikasi atau program semakin mudah karena tidak perlu membuat ADT Queue sendiri.

C. ReverseData_2511533003

```
package pekan4_2511533003;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511533003 {

    public static void main(String[] args) {
        Queue<Integer> q_3003 = new LinkedList<Integer>();
        q_3003.add(1);
        q_3003.add(2);
```

```
        q_3003.add(3); // [1, 2, 3]
        System.out.println("Sebelum Reverse " + q_3003);
        Stack<Integer> s_3003 = new Stack<Integer>();

        while (!q_3003.isEmpty()) {
            // Q -> S
            s_3003.push(q_3003.remove());
        }

        while (!s_3003.isEmpty()) {
            // S -> Q
            q_3003.add(s_3003.pop());
        }

        System.out.println("Sesudah Reverse = " + q_3003); // [3, 2, 1]
    }
}
```

Code diatas merupakan kode dari program yang berguna dalam reverse element di dalam suatu Queue, menggunakan ADT stack sebagai pembantu yang mana data akan dikirim secara bergantian menggunakan konsep FIFO dan FILO untuk membuat elemen pada queue terbalik.

D. IterasiQueue_2511533003

```
package pekan4_2511533003;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511533003 {

    public static void main(String[] args) {
        Queue<String> q_3003 = new LinkedList<>();

        q_3003.add("Pratikum");
        q_3003.add("Struktur");
        q_3003.add("Data");
        q_3003.add("Dan");
        q_3003.add("Algoritma");

        Iterator<String> iterator_3003 = q_3003.iterator();
        while (iterator_3003.hasNext()) {
            System.out.print(iterator_3003.next() + " ");
        }
    }
}
```

Code diatas merupakan code java yang berfungsi untuk menyimpan data String ke dalam suatu Queue, lalu menggunakan Iterator serta looping untuk mencetaknya dan membuatnya menjadi kalimat yang kata katanya berurutan.

E. Tugas

a. Deskripsi Program

Mahasiswa diminta untuk mengimplementasikan struktur data Queue Antrian) menggunakan array dalam bahasa Java. Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO First In First Out).

b. Pengerjaan

Queue_2511533003

```
package pekan4_2511533003;


import java.util.Scanner;
import java.util.Stack;


public class Queue_2511533003 {


    int front_3003, rear_3003, size_3003;
    int max_3003;
    String queue_3003[];


    public Queue_2511533003(int capacity_3003) {
        this.max_3003 = capacity_3003;
```

```
    front_3003 = this.size_3003 = 0;
    rear_3003 = capacity_3003 - 1;
    queue_3003 = new String[this.max_3003];
}

boolean isFull_3003(Queue_2511533003 queue_3003) {
    return (queue_3003.size_3003 == queue_3003.max_3003);
}

boolean isEmpty_3003(Queue_2511533003 queue_3003) {
    return (queue_3003.size_3003 == 0);
}

void enqueue_3003() {
    if (isFull_3003(this)) {
        System.out.println("penuh");
        return;
    }

    Scanner sc_3003 = new Scanner(System.in);
    System.out.print("Masukkan nama pelanggan: ");
    String item_3003 = sc_3003.nextLine();

    this.rear_3003 = (this.rear_3003 + 1) % this.max_3003;
    this.queue_3003[this.rear_3003] = item_3003;
    this.size_3003 = this.size_3003 + 1;
```

```
    System.out.println("Data berhasil ditambahkan ke antrian");
}

String dequeue_3003() {
    if (isEmpty_3003(this)) {
        System.out.println("Antrian Kosong");
        return "kosong";
    }

    String item_3003 = this.queue_3003[this.front_3003];
    this.front_3003 = (this.front_3003 + 1) % this.max_3003;
    this.size_3003 = this.size_3003 - 1;

    System.out.println(item_3003 + " telah dilayani");

    return item_3003;
}

void display_3003() {
    if (this.isEmpty_3003(this)) {
        System.out.println("\nAntrian Kosong\n");
        return;
    }

    System.out.println("\nIsi antrian: \n");
    for (
```

```
        int i_3003 = this.front_3003;
        i_3003 < this.front_3003 + this.size_3003;
        i_3003++
    ) {
        System.out.println(" " + queue_3003[i_3003]);
    }
}

void reverse() {
    if (isEmpty_3003(this)) {
        System.out.println("Antrian Kosong");
        return;
    }

    Stack<String> s_3003 = new Stack<>();

    for (
        int i_3003 = this.front_3003;
        i_3003 < this.front_3003 + this.size_3003;
        i_3003++
    ) {
        s_3003.push(queue_3003[i_3003]);
    }

    for (
        int i_3003 = this.front_3003;
```

```
        i_3003 < this.front_3003 + this.size_3003;
        i_3003++
      ) {
        queue_3003[i_3003] = s_3003.pop();
      }

      this.display_3003();
    }
}
```

Code diatas merupakan Code ADT yang bernama Queue_2511533003, fungsi utama adalah sebagai struktur yang akan menyimpan nama orang orang di dalam sebuah antrian berdasarkan konsep FIFO, method yang tersedia diantaranya ada

1. isFull_3003

   Berfungsi untuk mengetahui apakah Queue sudah penuh atau belum.

2. isEmpty_3003

   Berfungsi untuk mengetahui Queue kosong atau belum.

3. enqueue_3003

   Berfungsi untuk memasukkan data ke dalam Queue dengan konsep FIFO.

4. dequeue_3003

   Berfungsi untuk mengeluarkan data ke dalam Queue dengan konsep FIFO.

5. display_3003

   Menampilkan data antrian yang tersimpan

6. reverse

   membalikkan Data yang ada di antrian.

Code ADT yang bernama Queue_2511533003 masih belum dapat dijalankan dikarenakan tidak memiliki method main, oleh karena itu diperlukan code sebagai Driver yaitu AntrianLoket_2511533003.

AntrianLoket_2511533003

```
package pekan4_2511533003;

import java.util.Scanner;

public class AntrianLoket_2511533003 {

    public static void main(String[] args) {
        Queue_2511533003 queue_3003 = new Queue_2511533003(100);

        Scanner sc_3003 = new Scanner(System.in);
        System.out.println("=== PROGRAM ANTRIAN LOKET ===");
        while (true) {
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. keluar");
```

```
            System.out.print("pilih menu: ");
            int option_3003 = sc_3003.nextInt();
            System.out.println("");
            if (option_3003 == 1) {
                queue_3003.enqueue_3003();
            } else if (option_3003 == 2) {
                queue_3003.dequeue_3003();
            } else if (option_3003 == 3) {
                queue_3003.display_3003();
            } else if (option_3003 == 4) {
                queue_3003.reverse();
            } else if (option_3003 == 5) {
                break;
            } else {
                System.out.println("Pilihan tidak tersedia pada opsi");
            }
            System.out.println("");
        }

        sc_3003.close();
    }
}
```

Code diatas adalah Code yang menggunakan ADT Queue_2511533003, pertama kali running akan menampilkan opsi yang telah disediakan yaitu:

```
arax@fedora:~/Documents/kuliah/semest
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu:
```

Memilih opsi satu akan diminta kepada user untuk memasukkan nama yang akan dimasukkan ke dalam antrean.

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 1

Masukkan nama pelanggan: yazem
Data berhasil ditambahkan ke antrian

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 1

Masukkan nama pelanggan: agva
Data berhasil ditambahkan ke antrian

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 1

Masukkan nama pelanggan: roiz
Data berhasil ditambahkan ke antrian

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu:
```

Untuk dapat melihat semua data yang ada di dalam antrean dapat menggunakan opsi 3.

```
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 3


Isi antrian:

 yazem
 agva
 roiz

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu:
```

Untuk melayani orang di antrian terdepan, dapat menggunakan opsi ke dua. Lalu bisa dilihat data yang tertinggal di dalam antrean menggunakan opsi ke-3.

```
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 3


Isi antrian:

 yazem
 agva
 roiz

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 2

yazem telah dilayani

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 3


Isi antrian:

 agva
 roiz

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu:
```

Gunakan opsi ke 4 untuk membalikkan data.

```
Isi antrian:

 agva
 roiz

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 4


Isi antrian:

 roiz
 agva

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu:
```

Lalu opsi ke-5 untuk keluar.

```
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. keluar
pilih menu: 5

arax@fedora:~/Documents
```

BAB 3

PENUTUPAN

a. Kesimpulan

Queue merupakan implementasi ADT yang penting dan berguna, digunakan pada banyak kasus serta menjadi acuan struktur data untuk penanganan FIFO.

b. Saran

Laporan praktikum yang telah penulis tuliskan, masih memiliki kekurangan. Penulis sendiri pun juga membuka saran dan kritikan untuk meningkatkan kualitas laporan pratikum ini.

DAFTAR PUSTAKA

Oracle. (2026). Queue (Java Platform SE 8). Diakses pada 29 April 2026, dari https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html